Ders 2 Genişletilmiş Notu: Toplayıcılar, İkiye Tümleyen Aritmetiği ve Taşma, Three-State Buffer, Flip-Flop'lar ve Ardışıl Devre Analizi

Ders 2 Genişletilmiş Notu: Toplayıcılar, İkiye Tümleyen Aritmetiği ve Taşma, Three-State Buffer, Flip-Flop'lar ve Ardışıl Devre Analizi

Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Çerçevesi: Yapı Taşları
- 2. Yarım Toplayıcı ve Tam Toplayıcı
- 3. Çok Bitli Yapı Taşları
- 4. Karşılaştırıcı (Comparator)
- 5. İkiye Tümleyenle İşaretsiz Çıkarma
- 6. İşaretli Sayılar ve Taşma
- 7. Üç Durumlu Tampon (Three-State Buffer)
- 8. Ardışıl Devreler: Latch ve Flip-Flop
- 9. Flip-Flop Karakteristik Tabloları
- 10. Lojik Eksiğini Kapatmak
- 11. Tersine Mühendislik: Mealy Model Bir Devrenin Analizi
- 12. Moore Model Devre

1. Dersin Çerçevesi: Yapı Taşları

Bu hafta lojik konularının tekrarı sürüyor. Geçen hafta Boole cebrinde kullanılan AND ve OR işlemleri ile bunların türevleri, lojik kapılar ve De Morgan kuralı hatırlatılmış, ayrıca decoder, multiplexer, demultiplexer ve encoder gibi devre elemanları üzerinde örnekler çözülmüştü. Bu hafta kombinasyonel devrelerle devam ediliyor, ardından ardışıl devrelere geçiliyor.

Hoca bu elemanlara "yapı taşı" diyor. Yarım toplayıcı ve tam toplayıcı gibi toplama devreleri de birer yapı taşıdır, çünkü bunlar bir araya getirilerek CPU içindeki aritmetik devre oluşturulacak. Yapı taşları başta iyi öğrenilirse, bunların birbirine entegre edildiği daha karmaşık çözümler kolay anlaşılır. Hoca bu konuların lojik devreler dersinde öğrenildiğini kabul ediyor. Bu dersteki amacı hızlı bir tekrar ve hatırlatmadır.

2. Yarım Toplayıcı ve Tam Toplayıcı

2.1 Yarım Toplayıcı (Half Adder)

Yarım toplayıcı, her biri bir bit olan A ve B girişlerini toplar. Bir toplam (sum) ve bir elde (carry out) çıkışı üretir. Doğruluk tablosu şöyledir:

A	B	Sum	Carry out
0	0	0	0

A	B	Sum	Carry out
0	1	1	0
1	0	1	0
1	1	0	1

Girişlerden yalnız biri 1 olduğunda toplam 1 olur. İkisi de 1 olduğunda toplam 0, elde biti 1 olur. Devre şemasında toplam çıkışı bir XOR kapısıyla gerçekleştirilir: $S = A \oplus B$. Elde çıkışı ise iki girişin de 1 olduğu tek satırda 1 olduğu için $C = AB$ olur.

Yarım toplayıcının sorunu, yalnız iki bit giriş almasıdır. Amacımız yalnız bir bitlik iki sayıyı toplamak değil, örneğin dört bitlik iki sayıyı toplamaktır. Toplama işleminde bir elde taşıma işi vardır ve bu elde bitinin bir sonraki basamağa aktarılması gerekir. Yarım toplayıcıda elde girişi olmadığı için onu genişleterek daha karmaşık bir devre kuramayız.

2.2 XOR Kapısını Bir Cümleyle Özetlemek

Hoca geçen haftaki bir soruyu tekrar sordu: XOR kapısının çalışmasını bir cümleyle nasıl özetleriz? Üstelik yalnız iki girişli XOR için değil, üç ya da dört girişli XOR için de geçerli bir cümle arıyordu.

Soru: XOR'u tek cümleyle nasıl özetleriz? Sınıftan "tek sayıda" diye bir cevap geldi, cevabı veren öğrenci sonra yanıldığını düşünüp vazgeçti. **Cevap:** Hoca "tek sayıda" ifadesinin doğru yönde olduğunu söyleyip cümleyi tamamladı: Girişlerdeki 1'lerin sayısı tekse sonuç 1 olur.

Yarım toplayıcının toplam çıkışı da tam bu nedenle iki girişli bir XOR'dur. Toplamın 1 olduğu iki satırda girişlerde tek sayıda 1 vardır.

2.3 Tam Toplayıcı (Full Adder)

Tam toplayıcı üç bit toplar: A ve B girişlerinin yanında bir de elde girişi (C_{in}) vardır. Sonuç olarak iki bit üretir: toplam ve elde çıkışı (C_{out}). Hoca tablonun, tam toplayıcının çalışma mantığı bilindiğinde rahatça yazılabileceğini söyledi. Derste tek tek okunmayan satırlar toplama tanımından tamamlandı (ara adım eklendi).

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Tasarım yolu şöyledir: önce doğruluk tablosu oluşturulur, sonra sadeleştirilmiş fonksiyonu bulmak için bir Karnaugh haritası kurulur. Fonksiyon gözle görülebiliyorsa devre doğrudan da çizilebilir.

Toplam çıkışına bakalım. Toplam yalnız girişlerdeki 1'lerin sayısı tek olduğunda 1'dir. Bu yüzden Karnaugh haritasına taşınsa sadeleşmez ve XOR ile kurulması gerektiği görülür. Slayttaki şemada iki tane iki girişli XOR kullanılmıştır, aynı iş üç girişli bir XOR ile de yapılabilirdi:

$$S = A \oplus B \oplus C_{in}$$

Slaytta bir de küçük bir alıştırma vardı: iki yarım toplayıcı ile bir tam toplayıcı yapmak. Hoca bunu lojikte yaptığımızı düşündüğü için üzerinde durmadı.

Ek bilgi (derste anlatılmadı): Hoca elde çıkışının denklemini okumadı. Kısa notlarda $C_{out} = AB + C_{in}(A \oplus B)$ denklemi yukarıdaki tablonun sekiz satırıyla python'da karşılaştırıldı ve tam uyuştu. Bu biçim iki yarım toplayıcı alıştırmasını da açıklar: birinci yarım toplayıcı $A \oplus B$ ve AB üretir, ikincisi $A \oplus B$ ile C_{in}'i toplar, iki elde çıkışı OR kapısında birleşir.

Hocanın vurgusu: Bundan sonraki çözümlerde elimizde hep hazır bir tam toplayıcı olduğu varsayılacak ve aritmetik devre tasarımı bu tam toplayıcıyla yapılacak.

2.4 Ripple Carry Adder

Bir tam toplayıcı yapısını çoklayarak n bitlik bir toplayıcı kurabiliriz. Bu n değeri 8 ya da 16 olabilir. Tam toplayıcılar yan yana getirilerek daha büyük sayılar toplanır. Tam toplayıcılarla oluşturulan bu n bitlik devreye **ripple carry adder** denir.

Ders sonrası not: Hoca elde bitinin "bir önceki bite" taşındığını söyledi; doğrusu bir üst, yani bir yüksek anlamlı bite taşınmasıdır. Hoca devre şemasını anlatırken de bunu "bir yüksek anlamlı bit" diye ifade etti.

3. Çok Bitli Yapı Taşları

İşlemci aritmetik işlemleri 8, 16 ya da 32 bitlik sayılarla yapar. Veri yolu (bus) genişliğinden söz ederken de "32 bit veri yolu" deriz. Bu nedenle şimdiye kadar öğrenilen bütün yapı taşlarını, multiplexer ve decoder'dan toplayıcılara kadar, çok bit için genişletmek gerekir.

3.1 8 Bitlik 4'ten 1'e Multiplexer

Grafik sembolde W_0'dan W_3'e dört giriş vardır. Kenardaki işaret her girişin 8 bit olduğunu gösterir, F çıkışı da 8 bittir. Çalışma mantığı bildiğimiz 4'ten 1'e multiplexer ile aynıdır. Fark, seçimin tek bit yerine 8 bitlik sayılar arasında yapılmasıdır. Seçme girişi 00 olduğunda 8 bitlik W_0 sayısı, 11 olduğunda W_3 sayısı F çıkışına gider.

Ders sonrası not: Hoca bir yerde "8 tane sayı arasında seçme" dedi; doğrusu 4 tane 8 bitlik sayı arasında seçmedir. Hoca birkaç dakika sonra da "elimizde 4 tane sayı vardı" diye anlattı.

Hoca sınıfa iç yapının nasıl kurulduğunu sordu.

Soru: Slayttaki iç yapıda yapılan şey nedir? **Cevap:** Bir öğrenci, bütün bitler için S_0 ve S_1'in aynı biçimde bağlandığını ve çıkışların en sonda birleştirildiğini söyledi. Hoca buna ekleme yaptı: Her multiplexer'ın girişinde aynı ağırlığa sahip bitler kullanılmıştır.

İç yapıda 8 tane 1 bitlik 4'ten 1'e multiplexer vardır. Dört sayının en düşük anlamlı bitleri bir multiplexer'a girer ve en düşük anlamlı çıkış F_0 oluşur. Dört sayının 7 ağırlıklı, yani en yüksek anlamlı bitleri ise başka bir multiplexer'a girer. Bütün multiplexer'ların seçme uçları ortaktır. Arka tarafta sekiz multiplexer olsa da devreyi tek bir grafik sembolle gösteririz.

3.2 16 Bitlik İşaretsiz Toplayıcı

Slayttaki blok şemada iki adet 16 bitlik giriş (X, Y), bir C_{in} girişi, 16 bitlik toplam çıkışı ve 1 bitlik C_{out} çıkışı vardır. C_{in} girişi, bu 16 bitlik yapıyı kaskat bağlayarak 32 bite taşımak istediğimizde kullanılır.

Bu bloğun iç yapısı ripple carry adder'dır. En sağdaki tam toplayıcı en düşük anlamlı A_0 ve B_0 bitlerini toplar ve en düşük anlamlı sonuç bitini (R_0) oluşturur. Onun elde girişi 0'a bağlıdır. Elde çıkışı ise bir yüksek anlamlı bitin, yani A_1 ve B_1'i toplayan tam toplayıcının elde girişine bağlanır ve R_1 oluşur. Devrenin kararlı ve doğru bir sonuç vermesi için eldenin en düşük anlamlı bitten en yüksek anlamlı bite kadar taşınması gerekir.

3.3 Toplayıcı-Çıkarıcı (Adder-Subtractor)

Hoca grafik sembole dikkat çekti: Her tam toplayıcıyı ayrı ayrı çizmek yerine tek bir "n bit full adder" bloğu çizilir. Girişlerden biri sola, diğeri sağa verilir, toplam aşağıda gösterilir. Bu sembol özellikle işlemci tasarımına doğru gidildiğinde farklı kaynaklarda karşımıza çıkacak.

Bu devreye n bitlik X ve Y girişi gelir. Devrenin hem toplama hem çıkarma yapabilmesi için Y girişi XOR kapıları üzerinden, bir kontrol ucuyla denetlenir. Kullanılan mantık XOR'un şu özelliğidir:

- XOR'un bir girişi 0 ise çıkış diğer girişle aynıdır.
- XOR'un bir girişi 1 ise çıkış diğer girişin tümleyenidir.

Aynı kontrol ucu toplayıcının elde girişine de bağlanmıştır. Kontrol ucuna S dersek iki durum vardır:

- $S = 0$: Y toplayıcıya doğrudan gider, $C_{in} = 0$ olur. Sonuç $X + Y$'dir.
- $S = 1$: Y'nin tümleyeni toplayıcıya gider, $C_{in} = 1$ olur. Sonuç $X + \overline{Y} + 1$'dir.

$\overline{Y}+1$, Y'nin ikiye tümleyenidir. X ile Y'nin ikiye tümleyeni toplandığında yapılan işlem $X-Y$ olur. Böylece tek devreyle toplama ve çıkarma arasında seçim yapılır.

4. Karşılaştırıcı (Comparator)

Karşılaştırıcı, n bitlik iki sayıyı karşılaştırır ve sonuçta bir bayrak (flag) üretir. Örneğin A ve B n bitlik sayılar olsun. $A > B$ ise çıkış 1, küçük ya da eşitse 0 olan bir devre isteyebiliriz.

İki bitlik ya da üç bitlik sayılar için bu iş doğruluk tablosuyla çözülür. Fakat 8 ya da 16 bitlik sayıları bir doğruluk tablosuna koyup çözemeyiz. Hoca burada sınıfa nasıl bir yol izlenebileceğini sordu.

Soru: n bitlik sayılarda büyüklük karşılaştırması nasıl yapılır? Bir öğrenci en önemli bitten başlamayı önerdi: En önemli bitten en önemsize doğru inilir, bitler eşitse sonrakine geçilir, ilk farklı bitte hangi sayınınki büyükse o sayı büyüktür. **Cevap:** Hoca bu yolun da doğru olduğunu söyledi. Ancak varmak istediği nokta çıkarma işleminin sonucuna bakmaktı. Elimizde aritmetik işlemler için zaten hazır bir modül vardır, bu yüzden tercih edilen çözüm odur. Bu çözüm Bölüm 5'te örnekle gösterildi.

4.1 Eşitlik Kontrolü

Eşitlik kontrolü daha kolaydır. Slaytta 4 bitlik bir örnek vardı. A ve B'nin aynı ağırlıktaki bitleri birer XOR kapısına girer. Bir XOR çıkışının 0 olması, o iki bitin birbirine eşit olduğunu gösterir. Dört XOR çıkışı son olarak bir NOR kapısına verilir:

$$F = \overline{(A_3 \oplus B_3) + (A_2 \oplus B_2) + (A_1 \oplus B_1) + (A_0 \oplus B_0)}$$

Bütün XOR çıkışları 0 ise OR kapısının çıkışı 0 olurdu. NOR kullanıldığı için çıkış 1 olur, yani $A = B$ sonucu çıkar. Dört XOR çıkışından biri bile 1 olursa eşitlik bozulur ve NOR çıkışı 0 olur.

5. İkiye Tümleyenle İşaretsiz Çıkarma

Hoca, $A > B$ karşılaştırmasının nasıl yapıldığını göstermek için ikiye tümleyen aritmetiğiyle bir alıştırma hazırlamıştı. Elimizde iki tabanında yazılmış 7 bitlik iki sayı vardır ve $X - Y$ işleminin sonucu isteniyor:

$$X = 1010100_2, \qquad Y = 1000011_2$$

Çıkarma yapılırken toplamadan yararlanılır: Y'nin ikiye tümleyeni alınır ve X ile toplanır.

$$X - Y = X + (Y\text{'nin ikiye tümleyeni})$$

5.1 İkiye Tümleyen Nasıl Alınır?

Soru: Bir sayının ikiye tümleyeni nasıl bulunur? Bir öğrenci bitleri çevirip "başına" bir eklemeyi önerdi. **Cevap:** Hoca "başına bir eklemek" ifadesini kesin bir dille reddetti. Bu ifade en yüksek anlamlı bite 1 eklemek gibi anlaşılır. Öğrenci kastettiğinin bitleri çevirip sayıya 1 eklemek olduğunu söyleyince hoca bunu kabul etti.

> **Hocanın vurgusu:** "Tersini çevirip başına 1 eklemek" en tehlikeli, en yanlış ifadedir. Doğrusu bitleri ters çevirip sayıya, yani en düşük anlamlı bite 1 eklemektir.

İki yol vardır. Sınıftan gelen bir başka önerideki kestirme yol da bunlardan biridir. Ayrıca "büyük sayıdan çıkarma" diye bir öneri daha geldi, hoca bunun üzerinde durmadı.

Uzun yol. Önce birler tümleyeni alınır, sonra 1 eklenir:

```
Y                 = 1000011
birler tümleyeni  = 0111100
1 eklenince       = 0111101
```

Kestirme yol. En sağdan başlanır, ilk 1 görülene kadar bitler aynen yazılır, ilk 1 de aynen yazılır. Geri kalan bitlerin tümleyeni alınır. $Y = 1000011$ sayısında en düşük anlamlı bit zaten 1'dir ve aynen yazılır. Kalan 100001 kısmının tümleyeni 011110 olur. Sonuç yine 0111101'dir.

5.2 Toplama ve Sonuç

X ile Y'nin ikiye tümleyeni toplanır:

```
X                =   1010100
Y'nin 2'ye tüml. = + 0111101
------------------------------
sonuç            = 1 0010001
                   ^
                   C biti
```

Sonuç 7 bittir: $0010001_2 = 17$. Soldan taşan bit ise elde bitidir, $C = 1$.

Kontrol edelim. X'in değeri $64 + 16 + 4 = 84$, Y'nin değeri $64 + 3 = 67$'dir. Demek ki 10 tabanında $84 - 67$ işlemini yapmış olduk ve sonuç gerçekten 17'dir.

5.3 Borç Biti ve Karşılaştırma

Burada ilginç bir nokta var. İşlem sonucunda bir C biti çıktı. Borç (borrow) biti, elde bitinin tümleyenidir:

$$B = \overline{C}$$

Bu örnekte $C = 1$ olduğu için $B = 0$'dır. Bu bize X'i Y'den çıkarmak için borç almaya gerek olmadığını söyler. Yani C bitinin 1, B bitinin 0 çıkması, X'in Y'den küçük olmadığının göstergesidir. Hoca bu örneği özellikle bu yüzden gösterdi: Bir karşılaştırıcı, bir aritmetik devre yardımıyla da oluşturulabilir.

> **Ders sonrası not:** Hoca $C = 1$ çıkmasını $X > Y$'nin göstergesi olarak anlattı; doğrusu $C = 1$ koşulu $X \geq Y$ demektir. $X = Y$ durumunda da $C = 1$ ve $B = 0$ çıkar. Bu, 7 ve 8 bitlik bütün sayı çiftleri için python ile denendi. Kesin büyüklük için ayrıca sonucun sıfır olmadığına bakmak gerekir.

6. İşaretli Sayılar ve Taşma

6.1 İşaretli Çıkarma: $+6 - (+13)$

İkinci örnekte sayılar işaret bitiyle birlikte verilmiştir: $X = +6$, $Y = +13$. Bu sayılar ikiye tümleyen kullanılarak 8 bitlik biçimde yazılacak ve aritmetik işlemler yapılacak.

$$X = +6 = 00000110, \qquad Y = +13 = 00001101$$

13 için $8 + 4 + 1$ yazılır. $X - Y$ işlemi için yine Y'nin ikiye tümleyeni alınır. En sağdaki bit 1 olduğu için aynen yazılır, kalan bitlerin tümleyeni alınır:

$$Y\text{'nin ikiye tümleyeni} = 11110011$$

Toplama şöyledir:

```
X  =   00000110
     + 11110011
---------------
       11111001
```

Sonuç 11111001'dir. İşaret biti 1 olduğu için negatif bir sayıdır ve değeri -7'dir (ara adım eklendi). Bu da $6 - 13 = -7$ ile uyuşur.

6.2 Taşma Biti (Overflow) ve $+6 + 13$

İşaretli sayılarla işlem yaparken taşma biti önem kazanır. Taşma biti, işaret bitine gelen elde ile işaret bitinden çıkan elde arasında XOR işlemi yapılarak bulunur:

$$V = C_n \oplus C_{n-1}$$

İki elde birbirinden farklıysa taşma vardır, aynıysa taşma yoktur.

> **Ders sonrası not:** Hoca "çıkarma işleminde overflow oluşmaz" dedi; doğrusu aynı işaretli iki sayının farkında taşma olmaz, farklı işaretli iki sayının farkında olabilir. 8 bitlik bütün sayı çiftleri python ile denendi ve çıkarmada $V = 1$ durumunun yalnız farklı işaretli sayılarda çıktığı görüldü. Kısa nottaki $100 - (-100) = 200$ örneği de sonucun 8 bitlik aralığa sığmadığını gösterir: İşaret bitine giren elde 1, çıkan elde 0'dır, $V = 1$. Çıkarma devrede bir toplama olarak yapıldığı için V biti de aynı XOR ile hesaplanır. Hocanın aranın ardından gösterdiği devre şeması da bu XOR'u toplayıcı-çıkarıcı üzerinde kullanıyordu. Bu örnekteki $+6 - 13$ işleminde ise iki sayı da pozitif olduğu için taşma yoktur.

Hoca taşma kontrolünü bu nedenle aynı iki sayının toplamı üzerinde gösterdi. İki sayı da oldukça küçük olduğu için taşma beklenmiyor.

```
X  =   00000110
Y  = + 00001101
---------------
       00010011
```

Sonuç $16 + 3 = 19$'dur. İşaret bitine gelen elde 0'dır, işaret bitinden dışarı da elde çıkmamıştır. $V = 0 \oplus 0 = 0$, yani taşma yoktur.

6.3 İkiye Tümleyen Gösterimi

Hoca taşma örneğine geçmeden önce işaretli sayıların nasıl gösterildiğini anlattı. İşaretli sayıları temsil etmek için ikiye tümleyen yapısı kullanılır. Bilgisayar ve işlemci yapılarının tamamında işaretli sayılar için bu yapı geçerlidir. 4 bitlik gösterim şöyledir. Derste okunmayan satırlar python ile üretildi (ara adım eklendi).

Sayı	4 bit	Sayı	4 bit
$+7$	0111	-1	1111
$+6$	0110	-2	1110
$+5$	0101	-3	1101
$+4$	0100	-4	1100
$+3$	0011	-5	1011
$+2$	0010	-6	1010
$+1$	0001	-7	1001
0	0000	-8	1000

Sayı	4 bit	Sayı	4 bit

Bu tablodan çıkan noktalar şunlardır:

- En yüksek anlamlı bit işaret bitidir. Pozitif sayılarda 0, negatif sayılarda 1'dir.
- İkiye tümleyende $+0$ ve -0 ayrımı yoktur.
- 0 da pozitif bir sayı gibi görünür. Pozitif tarafta toplam 8 kombinasyon vardır, ama biri 0'a ayrıldığı için en büyük pozitif sayı $+7$ olur.
- Negatif tarafta en küçük sayı -8'dir ve 1000 ile gösterilir.

-1'in gösterimi şöyle bulunur: $+1 = 0001$, tümleyeni 1110, buna 1 eklenince 1111 olur.

> **Hocanın vurgusu:** -1'in ikiye tümleyen gösteriminde bütün bitler 1'dir. Bu aklınızda kalsın, işlemci içindeki aritmetik birim tasarlanırken sık karşımıza çıkacak.

8 bitlik bir register'da 256 farklı kombinasyon vardır. Bunun yarısı pozitif, yarısı negatif tarafa düşer. Böylece gösterilebilen aralık -128 ile $+127$ arasıdır.

6.4 Taşma Üreten Örnek: $+70 + 80$

Hoca işaretli sayılarda taşma üreten bir örnek verdi. Sayılar 8 bitle ifade edilecek. Sonucun 8 bitlik register'a sığmaması taşma demektir.

```
+70  =   01000110
+80  = + 01010000
-----------------
          10010110
```

İki pozitif sayı topladık ama sonuç negatif görünüyor. Sonuç bitleri işaretli olarak okunursa değeri -106 olur (ara adım eklendi). Doğru sonucu ifade edebilmek için 9. bite ihtiyaç vardır, register ise 8 bitliktir. Bu duruma taşma denir.

İşlemci bunu nasıl anlar? Toplama yapılırken işaret bitine gelen eldeye bakılır:

1. 6 ağırlıklı bitte $1 + 1 = 2$ olur. Sonuç biti 0, elde 1'dir. Bu elde işaret bitine girer: $C_7 = 1$.
2. İşaret bitinde $0 + 0 + 1 = 1$ olur. Dışarı elde çıkmaz: $C_8 = 0$.
3. $V = C_8 \oplus C_7 = 0 \oplus 1 = 1$.

Toplama devresindeki XOR kapısının çıkışı 1 olduğu için taşma sinyali üretilir.

> **Ders sonrası not:** Hoca "işaret biti dışarı taştı" dedi; doğrusu işaret bitine 1 elde girdi, işaret bitinden dışarı elde çıkmadı. Sonucun bitleri 8 bite sığdı ama işaret biti yanlış çıktı. Hocanın aynı örnekteki hesabı da bunu gösteriyor: $C_8 = 0$, $C_7 = 1$.

Aynı sonuca 10 tabanından da varılır: $70 + 80 = 150$. 8 bitlik register'da işaretli sayı aralığı -128 ile $+127$ arasındadır. 150 bu aralığın dışında kaldığı için taşma oluşur. Ancak işlemci işlemleri ikili yapar, bu yüzden taşma bitinin bir devre elemanıyla, yani bir XOR kapısıyla üretilmesi gerekir.

6.5 Durum Bitleri: C, V, Z, N

İşaretli sayılarla işlem yaparken taşma (V), sıfır (Z) ve negatif (N, işaret biti) bitleri anlamlıdır. İşlemci bu bayrakların hepsini bir devre elemanıyla üretir. Hangisinin dikkate alınacağına ise biz karar veririz. Karar ölçütü, sayıların işaretli ya da işaretsiz oluşudur.

- **İşaretsiz sayılar:** Yalnız C bitine bakılır. Bu bit, toplayıcı devresinin elde çıkışıdır.
- **İşaretli sayılar:** V, Z ve N bitlerine bakılır.

> **Hocanın vurgusu:** İşaretsiz sayılarla işlem yapılmışsa overflow biti kesinlikle kullanılmaz, yalnız carry biti önem kazanır.

Slaytta toplayıcı-çıkarıcı devresinin üzerinde bu yapı gösterilir: İşaret bitinden çıkan elde C_n, işaret bitine gelen elde C_{n-1} ile XOR'lanarak V biti üretilir. Toplayıcının elde çıkışı ise C bitidir.

Ek bilgi (derste anlatılmadı): Hoca Z ve N bitlerinin nasıl üretildiğini anlatmadı. Bu bitler büyük olasılıkla şöyle tanımlıdır: Z, sonucun bütün bitleri 0 olduğunda 1 olur. N ise sonucun en yüksek anlamlı bitidir.

7. Üç Durumlu Tampon (Three-State Buffer)

Ardışıl devrelere geçmeden önce hoca three-state buffer'ı anlattı. Bu eleman özellikle işlemci içinde veri yolu ve adres yolu gibi bus yapılarını kurarken kullanılır. Lojikte öğrenilmemiş olabilir.

7.1 Çalışması

Buffer, bir işareti bir yerden bir yere taşıyan, geçiren devre elemanıdır. Three-state buffer, X girişini F çıkışına aktarır, ama bunu bir enable (E) ucunun denetiminde yapar. Hocanın benzetmesiyle, girişteki işareti çıkışa ileten bir kanal gibi düşünülebilir. Bu kanalın ne zaman açık, ne zaman kapalı olduğuna enable ucu karar verir.

E	X	F
0	0	Hi-Z
0	1	Hi-Z
1	0	0
1	1	1

Üç durum 0, 1 ve Hi-Z'dir. Hi-Z yüksek empedanslı, yani yüksek direnç gösteren çıkış demektir. Z burada yüksek direnci gösterir.

7.2 Neden Gerekli? Çıkışları Birbirine Bağlamak

Normal kapıların çıkışları birbirine bağlanamaz. Örneğin bir AND kapısıyla bir OR kapısının çıkışını aynı hatta bağladığımızı düşünelim. AND çıkışı 1, OR çıkışı 0 ise bir taraf 5 volt, diğer taraf 0 volttur. 5 volttan 0 volta doğru akım akar ve hocanın deyişiyle alttaki OR kapısından dumanlar çıkar.

> **Hocanın vurgusu:** İki lojik devre elemanının çıkışını birbirine bağlamak yasaktır. Yapılabilecek tek şey, iki çıkışı alıp başka bir devre elemanına giriş olarak vermektir. Three-state buffer'larda ise çıkışlar birbirine bağlanabilir. Bu çok önemli bir özelliktir.

Şart şudur: Aynı anda yalnız bir buffer etkin olmalı, diğerlerinin çıkışı Hi-Z olmalıdır. İki buffer için bu, enable uçlarından birini bir inverter üzerinden sürerek garanti edilebilir. Böylece birinin enable ucu 1 iken öbürününki 0 olur. Hoca bu bağlantıya "kısa devre etmek" diyor, kastettiği iki çıkışı tek hatta bağlamaktır.

Üstteki buffer'ın enable ucu 0, alttakinin 1 olsun. Alttakinin girişinde de 1 olsun, çıkışı 1, yani +5 volt olur. Üstteki buffer Hi-Z olduğu için bu hattan içeri akım akmaz. Hi-Z olması, hattın kopuk olduğu, hiç devreye bağlı olmadığı gibi algılanır. Böylece üstteki buffer'ın yanması, üstünden dumanlar çıkması söz konusu olmaz.

Soru: Akımı mı düşürüyor? **Cevap:** Hoca akımın her zaman direncin düşük olduğu kola doğru aktığını hatırlattı. Alttaki buffer'ın çıkışında lojik 1, yani +5 volt vardır. Üstteki buffer yüksek direnç gösterdiği için akım onun içine doğru akamaz. Üstteki buffer, çıkışında Hi-Z üreterek korumaya alınmış olur.

İki buffer yerine sekizi de bir arada bağlanabilir. Bunun şartı, sekizinden yalnız birinin etkin, diğerlerinin hepsinin devre dışı olmasını garanti etmektir. O zaman sekiz buffer'ın çıkışı tek çıkış olarak kullanılabilir.

Soru: Three-state buffer'ı anlamadım. **Cevap:** Hoca kanal benzetmesiyle tekrar anlattı: Enable ucu 0 iken çıkışta yüksek direnç vardır, 1 olduğunda kanal açılır ve girişteki işaret çıkışa aktarılır. Yüksek direnç sayesinde birden fazla buffer'ın çıkışı birbirine bağlanabilir.

Soru: Bilgisayar donanımında nerede kullanılıyor? **Cevap:** Bus yapısını, yani işlemcinin veri yolunu oluşturmak için kullanılıyor. Farklı register'lardan gelen bir sayıyı veri yolunda göstermek için bu yapıya başvuruluyor. Hoca bus örneklerini ilerleyen haftalarda anlatacağını söyledi.

> **Ders sonrası not:** Hoca 8 bitlik veri yolu için 8 tane three-state buffer dedi; doğrusu bu sayı tek bir kaynak içindir. Hoca büyük olasılıkla tek bir register'ın bağlantısını kastetti. Her kaynak register'ın her biti için bir buffer gerekir, her bit hattında aynı anda o hatta bağlı buffer'lardan yalnız biri etkin olur.

7.3 Dört Türü

Slaytta dört tür three-state buffer vardır:

Tür	Etkin olduğu E	Etkinken F	Etkin değilken F
Temel form	1	X	Hi-Z
Evirici çıkışlı	1	$\overline{X}$	Hi-Z
Aktif düşük enable	0	X	Hi-Z
Aktif düşük enable, evirici çıkışlı	0	$\overline{X}$	Hi-Z

> **Hocanın vurgusu:** Bu türleri gördüğünüzde doğruluk tablosu verilmese bile nasıl çalıştığını tahmin edebilmelisiniz. Standart grafik sembollerdeki işaretler bir ipucu verir: Bu buffer enable ucu 1 iken çalışır, şu 0 iken çalışır, bunun çıkışında NOT kapısı var gibi yorumlar yapabilmelisiniz.

8. Ardışıl Devreler: Latch ve Flip-Flop

Böylece kombinasyonel kısım bitti. Dersin ikinci bölümü ardışıl devre tasarımıdır ve ardışıl devre elemanı olarak flip-flop kullanılacak.

8.1 Ardışıl Devrenin Yapısı

Ardışıl devrede bir kombinasyonel lojik bloğu ve durum tutan elemanlar (state holding element), yani flip-floplar bulunur. Flip-floplar bir clock işaretiyle çalışır. Clock bir kare dalgadır. Her periyodu t, $t+1$, $t+2$ diye adlandırabiliriz ve bu periyotların her biri bir durum (state) olarak geçer. $S(t)$ mevcut durumu, $S(t+1)$ bir sonraki periyotta olacak durumu gösterir. Devre tasarımı bu iki bloğu, kombinasyonel lojik ile flip-flopları, bir arada kullanarak yapılır.

Ardışıl devre tasarımında latch ve flip-flop diye iki ayrı yapı vardır.

8.2 SR Latch

SR latch'te S ve R adları set ve reset'ten gelir.

Clock	S	R	$Q(t+1)$	Anlamı
0	fark etmez	fark etmez	$Q(t)$	değişiklik yok
1	0	0	$Q(t)$	değişiklik yok
1	0	1	0	reset
1	1	0	1	set
1	1	1	tanımsız	kullanılmaz

Clock 0 iken S ve R ne olursa olsun bir önceki durum korunur. Clock 1 iken eleman etkin olur ve girişlere bakılır. S ve R'nin ikisi birden 1 ise tanımsız bir durum oluşur, diğer kombinasyonlar kullanılabilir. Standart grafik sembolü, elemanın clock işaretinin seviyesine bağlı olduğunu, yani clock 1 gördüğü süre boyunca işlem yaptığını gösterir.

8.3 D Latch

Benzer bir eleman da D tipi latch'tir. Clock 0 iken D ne olursa olsun mevcut değer tutulur: $Q(t+1) = Q(t)$. Clock 1 iken D girişinde ne varsa çıkışta o görülür: $Q(t+1) = D$.

Hoca bunu zaman diyagramı üzerinde anlattı. Clock 1 iken D'deki değişimlerin hepsi Q'da görülür, Q D ile paralel gider. D clock 0 iken değişirse Q değişmez, mevcut durumunu göstermeye devam eder. Clock 0'a düştüğü anda latch çıkışında bir önceki durumda ne varsa o görünmeye devam eder. Bu şekilde yalnız clock 1 iken işlem yapan elemana latch diyoruz.

8.4 D Flip-Flop

İşlemci içinde ve ardışıl devre tasarımında kullandığımız eleman latch değil, flip-floptur. Flip-flopta standart grafik sembol değişir: Clock girişindeki üçgen işareti pozitif kenar tetiklemeyi (positive edge trigger) gösterir. Pozitif kenar, clock'un 0'dan 1'e geçiş anıdır. Eleman yalnız bu anlarda değişim gösterir.

Zaman diyagramında clock'un pozitif kenarı geldiği anda D girişinde ne varsa Q çıkışına aktarılır. Clock 1 kaldığı süre boyunca D değişse bile Q değişmez. Latch ile fark budur: Latch clock 1 olduğu sürece işlem yapabilir, flip-flop yalnız kenarda işlem yapar.

Yalnız pozitif kenar tetiklemeli değil, negatif kenar tetiklemeli flip-floplar da vardır. Bunlar clock girişindeki üçgenin önüne konan bir işaretle (kısa nota göre küçük bir daire) gösterilir.

> **Hocanın vurgusu:** Devre tasarımında kullanacağımız eleman, kenar tetiklemeli flip-floptur.

8.5 Asenkron Preset ve Clear

Flip-flopların ayrıca asenkron preset ve clear girişleri olabilir. Preset öncelikli set etme, clear ise temizleme, yani sıfırlama demektir. Slayttaki flip-flopta bu uçların aktif seviyesi 0'dır.

Giriş	Değer	Sonuç
Preset	0	Set edilir, $Q = 1$
Preset	1	Devre dışı, normal çalışma
Clear	0	Temizlenir, $Q = 0$
Clear	1	Devre dışı, normal çalışma

> **Hocanın vurgusu:** Bu girişlerin asenkron olması çok önemlidir. Asenkron, clock'tan bağımsız çalışmak demektir.

Flip-flop normalde yalnız pozitif kenarda değişir. Asenkron preset ve clear ise pozitif kenarı beklemez. Bu uçlara aktif işaret geldiği anda set ya da reset gerçekleşir. Slayttaki zaman diyagramında clear girişi 0 olduğunda clock'un pozitif kenarları beklenmeden Q hemen sıfırlanır.

Senkron clear ise D girişi üzerinden yapılır. Clear işareti bir AND kapısıyla D ile birleştirilir ve kapının çıkışı flip-flopun D girişine bağlanır. Clear 0 iken D girişine 0 gider, Q ise yalnız clock'un pozitif kenarında değişir. Kısaca asenkron clock'tan bağımsız, senkron clock'a bağımlı çalışır.

Soru: Preset ve clear aynı anda etkin olursa ne olur? **Cevap:** İkisi birden 0 gelirse devre düzgün çalışmaz. SR latch'teki tanımsız durum gibi bu kombinasyon da verilmemelidir.

Soru: Senkron kullanımda reset nasıl bağlanır? **Cevap:** Clear işlemi ya flip-flopun kendi clear ucundan ya da D girişinin önündeki AND bloğundan verilebilir. Birincisi asenkron, ikincisi senkron olur. İkisi de kullanılabilir.

Soru: D ile clear'ı AND'leyince senkron oluyor. Preset ve clear'ın ikisini birden senkron yapmak istersek ne olur? **Cevap:** Hoca, preset ve clear uçlarının bu flip-flopun üzerinde fiziksel olarak bulunduğunu ve istenirse asenkron kullanılabileceğini söyledi. Senkron bir clear mekanizması istenirse AND kapısıyla kurulabilir. Cevapta senkron preset için ayrı bir devre verilmedi.

9. Flip-Flop Karakteristik Tabloları

Register konusuna girmeden önce flip-floplarla biraz daha çalışmak gerekiyor. Hoca hatırlatma için yaygın kullanılan elemanların karakteristik tablolarını yazdı. Hepsinde clock girişi vardır, flip-flopların Q ve $\overline{Q}$ çıkışları vardır.

9.1 D Flip-Flop

D	$Q(t+1)$	Anlamı
0	0	reset
1	1	set

9.2 SR Latch

Hoca önce "SR flip-flop" dedi ve bunun çok yaygın kullanılmadığını belirtti. Sonra verdiği elemanın latch örneği olduğunu söyledi. Ayrım sembolden anlaşılır: Clock girişinde kenar tetikleme üçgeni varsa flip-flop, yoksa latch'tir.

S	R	$Q(t+1)$	Anlamı
0	0	$Q(t)$	değişiklik yok
0	1	0	reset
1	0	1	set
1	1	tanımsız	

9.3 JK Flip-Flop

JK çok kullanılacak flip-floplardan biridir.

J	K	$Q(t+1)$	Anlamı
0	0	$Q(t)$	değişiklik yok
0	1	0	reset
1	0	1	set
1	1	$\overline{Q(t)}$	tümleyen

> **Hocanın vurgusu:** Bu karakteristik tabloları ezbere yazabilmelisiniz. Ardışıl devre tasarımında bunlardan bir de geçiş tablosu üretmeniz gerekecek.

Geçiş tablosu, $Q(t)$ ve $Q(t+1)$ biliniyorken girişlerin ne olması gerektiğini söyler. 00, 01, 10 ve 11 kombinasyonlarının her biri için J ve K yazılır. Karakteristik tablo biliniyorsa bu tablo üretilebilir. Hoca bunu devre tasarımı sırasında kullanacaklarını ve anlatacağını söyledi, tabloyu bu derste doldurmadı.

> **Ek bilgi (derste anlatılmadı):** Karakteristik tablodan üretilen JK geçiş tablosu python ile denendi: $Q(t)Q(t+1) = 00$ için $JK = 0X$, 01 için $JK = 1X$, 10 için $JK = X1$, 11 için $JK = X0$'dır. X, o girişin 0 ya da 1 olabileceğini gösterir. Örneğin Q'yu 0'dan 1'e geçirmek için $J = 1$ gerekir, K ne olursa olsun geçiş olur: $JK = 10$ set eder, $JK = 11$ tümleyen alır.

9.4 T Flip-Flop

T	$Q(t+1)$	Anlamı
0	$Q(t)$	değişiklik yok
1	$\overline{Q(t)}$	tümleyen

> **Ders sonrası not:** Hoca, altyazıya göre T girişi 0 iken "no change, yani QT'nin komplementi" dedi; doğrusu $T = 0$ ise $Q(t+1) = Q(t)$, $T = 1$ ise $Q(t+1) = \overline{Q(t)}$'dir.

Hoca, lojikte bir JK flip-floptan T ya da D tipi flip-flop üretmekle ilgili bol alıştırma yapıldığını tahmin ettiğini söyledi.

10. Lojik Eksiğini Kapatmak

İkinci aradan sonra hoca, konu tekrarı olduğu için hızlı gidildiğini ama örneklerle açıklamaya çalıştığını söyledi. Bu konularda eksiği olanlara, özellikle lojik devre geçmişi olmayan bilimsel hazırlık öğrencilerine, lojik konularını hızlıca tekrar etmelerini tavsiye etti. Hocaya göre konular zor değildir, bir hafta üzerinden geçilerek hızlıca öğrenilebilir.

Soru: Lojik dersini almış olmama rağmen aklımda çok şey kalmadı. Kapıları temelden anlatan Türkçe bir kitap var mı? Flip-flopun açma kapama mantığını ve iç yapısını da tam çözebilmiş değilim. **Cevap:** Bilgisayar mühendisliği öğrencisi olarak kapıların ya da flip-flopların nasıl çalıştığını devre düzeyinde anlamanız gerekmiyor. Hoca "elinizde böyle bir malzeme var, doğruluk tablosu da budur" diyor ve bu malzemeyle tasarım yapabilmenizi bekliyor. Bunun için bol örnek yapılacak.

Kaynak olarak Morris Mano'nun kitabını önerdi: Dili oldukça güzeldir, içinde çok örnek vardır ve hocanın bildiği kadarıyla Türkçesi de vardır. Dersin kitabı olan *Logic and Computer Design Fundamentals*'ın ilk 5-6 bölümü de lojik konularını özetler ve başlamak için yeterlidir.

> **Hocanın vurgusu:** Lojik tekrarını dersin ilk 2-3 haftasında, olabildiğince erken bitirin. Yoksa ilerleyen kısımlarda anlatılanları anlamak zorlaşır.

11. Tersine Mühendislik: Mealy Model Bir Devrenin Analizi

Mano'nun kitabından alınan bu örnekte tersine mühendislik (reverse engineering) yapılıyor. Bize bir problem verilmemiş. Birileri problemi incelemiş, çözümü düşünmüş, uygun bir devre tasarlamış ve devreyi önümüze koymuş. Problemden çözüme gitmek yerine çözümden, yapılan işi anlamaya çalışacağız. Konu tekrarı olduğu için böyle bir yol seçildi.

Devrede iki D tipi flip-flop (A ve B), önlerinde kombinasyonel devreler, bir X girişi ve bir Y çıkışı vardır.

Soru: Bu devre şemasına bakarak ilk ne yapabilirim? **Cevap:** Sınıftan gelen cevap çıkışlardan başlamaktı. Bu, her flip-flop girişi ve Y çıkışı için birer fonksiyon yazmak demektir. Ardından bu fonksiyonlardan state table, oradan da state diagram çıkarılır.

11.1 Adım 1: Fonksiyonları Yazmak

Hoca denklemleri chat üzerinden sınıfa yazdırdı.

- D_A: Bir AND kapısı A ile X'i, ikinci AND kapısı B ile X'i AND'ler. Bir OR kapısı bu ikisini toplar.
- D_B: Bir AND kapısı $\overline{A}$ ile X'i AND'ler.
- Y: Bir OR kapısı A ile B'yi OR'lar. Çıkış, bir NOT kapısından geçen X ile AND'lenir.

$$D_A = AX + BX, \qquad D_B = \overline{A}X, \qquad Y = (A+B)\overline{X}$$

11.2 Adım 2: State Table

Devre kombinasyonel olsaydı doğruluk tablosu diyecektik. Flip-flop içeren ardışıl bir devre olduğu için buna state table denir. Solda mevcut durum (A, B) ve giriş (X), sağda sonraki durum ve çıkış (Y) yazılır. Üç giriş için 8 kombinasyon vardır. D flip-flopta sonraki durum giriş fonksiyonuna eşittir: $A(t+1) = D_A$, $B(t+1) = D_B$.

Hoca ilk satırları fonksiyonlardan tek tek çıkardı:

- $AB = 00$, $X = 0$: X 0 olduğu için D_A'nın iki terimi de 0'dır, D_B de 0'dır. Sonraki durum 00'dır. $\overline{X} = 1$ olsa da A ve B 0 olduğu için $Y = 0$'dır.
- $AB = 00$, $X = 1$: A ve B 0 olduğu için $D_A = 0$ olur. $D_B = \overline{A}X = 1$ olur. $\overline{X} = 0$ olduğu için $Y = 0$'dır.
- $AB = 01$, $X = 0$: Sonraki durum 00'dır. $A + B = 1$ ve $\overline{X} = 1$ olduğu için $Y = 1$'dir.

Tablonun tamamı şöyledir:

A	B	X	$A(t+1)$	$B(t+1)$	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

11.3 Adım 3: State Diagram

Dört durum (00, 01, 10, 11) birer düğüm olarak yazılır. Yerleşim önemli değildir. Her satır bir ok olur. Okun üzerine önce X, bölü işaretinden sonra Y yazılır.

Mevcut durum	Ok etiketi (X/Y)	Sonraki durum
00	0/0	00 (aynı durumda kalır)
00	1/0	01
01	0/1	00
01	1/0	11
10	0/1	00
10	1/0	10 (aynı durumda kalır)
11	0/1	00
11	1/0	10

Hoca devrenin hangi problemi çözdüğünü söylemedi. Burada amaç analiz adımlarını göstermekti.

Buna tersine mühendislik denmesinin nedeni izlenen sıradır. Problem baştan verilseydi yol şöyle olurdu: state diagram, state table, fonksiyonlar, devre. Burada ise devreden başlandı, önce fonksiyonlar yazıldı, sonra state table ve state diagram çıkarıldı.

11.4 Mealy Modeli

Y çıkış fonksiyonunun içinde X girişi yer alıyorsa devreye **Mealy model** devre denir. Çıkış girişe bağlı olduğu için state diagram çizilirken her geçişte girişe bağlı olarak çıkışın ne olduğu yazılır, yani X/Y biçimi kullanılır.

12. Moore Model Devre

Bu örnekte bize doğrudan devre fonksiyonu verilmiştir. Devrede bir D tipi flip-flop (A), iki giriş (x, y) ve bir z çıkışı vardır. Flip-flop giriş denklemi ve çıkış denklemi şöyledir:

$$D_A = A \oplus x \oplus y, \qquad z = A$$

Devrenin modeline karar verirken çıkışa bakılır. z çıkışı yalnız durum değişkeniyle oluşturulmuştur, fonksiyonunda x ve y girişleri yer almaz. Bu nedenle bu devre bir **Moore model** devredir. Çıkış yalnız duruma bağlı olarak değişir, girişlerden bağımsızdır.

Şemada D flip-flopun girişi üç girişli bir XOR ile sürülür, z çıkışı flip-flopun A çıkışıdır. State table'ı yukarıdaki fonksiyona göre, 8 kombinasyon için doldurulur:

A	x	y	$A(t+1)$	z
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	0
1	0	0	1	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Tek flip-flop olduğu için iki durum vardır: 0 ve 1. Moore state diagram'ında çıkış okun üzerinde değil, durumun içinde yazılır. Dairelerin içinde "durum/çıkış" bulunur, burada $0/0$ ve $1/1$'dir. Okların üzerinde yalnız girişler bulunur. Yukarıdaki tablodan geçişler şöyledir:

- $0/0$ durumunda: $xy = 00$ ya da 11 ise aynı durumda kalınır, $xy = 01$ ya da 10 ise $1/1$ durumuna geçilir.
- $1/1$ durumunda: $xy = 00$ ya da 11 ise aynı durumda kalınır, $xy = 01$ ya da 10 ise $0/0$ durumuna geçilir.

Soru: Mealy model için çıkış fonksiyonunda bütün girişlerin olması gerekiyor mu? **Cevap:** Hayır. Girişlerden yalnız birinin bulunması bile devreyi Mealy model yapar. Moore modelde ise çıkış fonksiyonunda hiçbir giriş değişkeni olmaz, çıkış yalnız durum değişkenlerine bağlıdır.

İki model state diagram'dan da ayırt edilir:

	Mealy	Moore
Çıkış fonksiyonu	En az bir giriş içerir	Yalnız durum değişkenleri içerir
Çıkışın yazıldığı yer	Okun üzerinde (X/Y)	Durumun içinde (durum/çıkış)
Okun üzerinde	Giriş ve çıkış	Yalnız giriş

Dersin Toparlaması

Ders, kombinasyonel yapı taşlarının çok bitli hâlleriyle başladı. Tam toplayıcılardan ripple carry adder, ona eklenen XOR kapıları ve kontrol ucuyla toplayıcı-çıkarıcı kuruldu. İkiye tümleyen aritmetiği bu devrenin çalışma mantığını açıkladı: Çıkarma bir toplamadır, elde biti işaretsiz sayılarda karşılaştırma bilgisi verir, işaret bitine giren ve çıkan eldenin XOR'u işaretli sayılarda taşmayı gösterir. Three-state buffer, işlemci içindeki veri yollarının kurulmasını sağlayan malzeme olarak tanıtıldı.

İkinci bölümde latch ile flip-flop ayrıldı, asenkron ve senkron girişler karşılaştırıldı, D, SR, JK ve T tabloları hatırlatıldı. Bir Mealy ve bir Moore devresi analiz edilerek devreden state table ve state diagram çıkarma adımları görüldü. Hoca bir sonraki hafta ardışıl devre tasarımında sayıcıların ve register'ların nasıl oluşturulduğunu tekrar edeceklerini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.